

GİT VE GİTHUB · 45 DAKİKA

Git ve GitHub

Birlikte üretmenin mantığı.
Değişikliklerimizi neden ve nasıl
paylaşırız?

Arif Solmaz
İSTÜN Mekatronik Mühendisliği



Değişen projeler ve kişisel hafıza

Dosyanın son hâli, oraya nasıl geldiğimizi anlatmaz.

- Kod değişir. Ürünün davranışı değişir.
- Deney değişir. Sonucun açıklaması değişir.
- Ekip değişir. Bilgiyi taşıyan insanlar ayrılır.

Tek başına: geçmişini bulabilmek

Kendi projenin yarınki çalışanı da sensin.

Hatırlamak

Neyi, neden değiştirdiğini
yeniden görebilirsin.

Karşılaştırmak

Çalışan hâle yeni hâl arasındaki
farkı inceleyebilirsin.

Devam etmek

Kaydettiğin bir hâli başlangıç
noktası yapabilirsin.

Hangi dosyadan devam edeceğiz?

SORUN

Hangisi son hali?

23:47

Robot Projesi

.ino

robot.ino

.ino

robot_v2.ino

.ino

robot_v2_deniz.ino

.ino

robot_final.ino

.ino

robot_final_SON.ino

.ino

robot_final_SON_kaan.ino

.ino

robot_final_GERCEKSON.ino

?

Robot Takımı

K

Kaan

Elif, son kodu atar mısın?

E

Elif

robot_final.ino'yu attım

robot_final.ino

D

Deniz

Ama ben sensörü robot_v2'de düzeltmişim...

K

Kaan

Robot yine çizgiden çıktı. Hangisi son hali??

E

Elif

Bilmiyorum. Hepsini tek tek mi deneyelim?

Kim neyi değiştirdi? Hangisi çalışıyordu?

Kimse bilmiyor.

Kopyalar arttıkça hangi hâle güveneceğimizi anlamak zorlaşır.

Sürüm kontrolü neyi kaydeder?

Bir çalışmanın farklı hâllerini ve aralarındaki ilişkiyi.

- Kaydedilmiş hâller
- Bu hâller arasındaki değişiklikler
- Yazar bilgisi ve değişiklik açıklamaları

Git: projenin geçmişini tutan araç

Depo (repository): dosyalar ve onları izleyen kayıtlar.

Proje dosyaları

Üzerinde çalıştığın kod, metin ve ilgili dosyalar.

Kaydedilmiş geçmiş

Projenin nasıl değiştiğini inceleyebileceğin kayıtlar.

Git kullanmak için GitHub hesabı gerekmez.

GitHub: değişikliği birlikte konuşmak

Git depolarını paylaşmak ve katkıları görüşmek için bir platform.

Git

Bilgisayarında değişiklik geçmişi tutar.

GitHub

Paylaşım, görüşme ve katkı incelemesi için ortak ortam sunar.

GitHub olmadan Git kullanılabilir. Başka platformlar da vardır.

Yerel çalışma ve paylaşım

Geçmişin yerel kopyası, çalışmayı bağlantıya bağlamaz.

- Yerel kopyanda geçmişçi görebilirsin.
- Çevrimdışıyken yeni kayıt oluşturabilirsin.
- Paylaşınca diğerlerinin çalışmasıyla buluşursun.

Commit: açıklamalı bir kayıt

Projenin belirli bir hâlini, bir açıklamayla kaydetmek.

Kayıt

Projenin belirli bir hâli saklanır.

Açıklama

Bu değişikliğin amacı görünür olur.

İlişki

Kayıt, önceki geçmişe bağlanır.

Geçmiş neyi kanıtlar?

Neyin değiştiğini gösterir. Değişikliğin doğru olduğunu garanti etmez.

Gösterir

Hangi dosyalar nasıl değişti. Hangi açıklama yazıldı.

Ayrıca doğrulanır

Kodun çalışması, ölçümün geçerliliği, kararın uygunluğu.

Küçük değişiklik, anlaşılır karar

Bir kaydın amacı netse incelemek ve devam etmek kolaylaşır.

- Birbiriyle ilgili değişiklikler aynı kayıttadır.
- Açıklamada yalnızca ne olduğu değil, neden olduğu.
- Sonradan okuyacak kişiye yetecek bağlam.

Branch: ayrı bir çalışma yolu

Dal (branch): aynı geçmişten farklı bir yönde ilerlemek.



- Bir fikir ayrı bir dalda gelişir.
- Ekip uygun gördüğünde ortak çalışmayla birleşir.

Dal, denemeye alan açar. Sonucu doğrulama görevini kaldırmaz.

Merge: çalışmaları buluşturmak

Birleştirme (merge): ayrı ilerleyişleri ortak geçmişe almak.

- Bağımsız değişiklikler otomatik birleşebilir.
- Aynı yere farklı kararlar gelirse seçim gerekir.
- Çatışma çıkmaması, sistemin doğru çalıştığını göstermez.

Takımda: birbirini durdurmada ilerlemek

Ayrı çalışabilmek, ortak karar verebilmekle tamamlanır.

Paralel çalışma

Farklı işler aynı zamanda ilerleyebilir.

Ortak karar

Katkı, amacı ve etkisi görülerek değerlendirilir.

Ortak hafıza

Konuşulanlar ilgili değişikliklerle birlikte bulunabilir.

İnsanlar bunu günlük işte nasıl kullanır?

Bir ihtiyacın ortak projeye nasıl katıldığını izleriz.

1. Ekip ihtiyacı ve beklenen sonucu açıklar.
2. Bir kişi ayrı dalda değişikliği hazırlar.
3. Ekip öneriyi ve doğrulama sonuçlarını inceler.
4. Ekip uygun değişikliği ortak projeye alır.

GitHub Flow bu çalışma biçiminin bir örneğidir.

Issue: konuşulabilir bir ihtiyaç

3 FAYDA

Takımın yapılacaklar listesi

E Elif'in ekranı

github.com/elif/robot-projem/issues

elif / robot-projem Public

<> Code Issues 2 Pull requests 0 Actions

Q is:issue state:open Labels New issue

2 Open 1 Closed Author Labels Assignee

Rapor için robotun fotoğrafını çek rapor
#3 opened 2 minutes ago by elif

Motor sürücüsü ısınıyor bug
#1 opened 2 days ago by kaan

SENİN İÇİN

Kim neyi yapıyor, herkes görür

✓ İşler mesaj grubunda kaybolmaz; aylar sonra bile bulunur.

✓ Her işin bir sorumlusu var.

✓ Bitirme projesi, kulüp, ödev grubu: hepsine uyar.

Issue = yapılacak bir iş ya da bir hata notu.

İş bitince **Close issue** : listeden düşer, kaydı kalır.

Issue, bir sorunun veya önerinin ortak konuşma ve takip yeridir.

Pull request: bir değişiklik önerisi

PR: “Bu çalışmayı ortak projeye alalım mı?”

Öneri

Ne değişti ve neden değişti?

Görüşme

Sorular, yorumlar ve düzeltme istekleri.

Karar

Ortak projeye alınır, bekletilir veya kapatılır.

Review: ikinci bir göz

İnceleme (review): değişikliği anlamak ve değerlendirmek.

- Amacı ve varsayımları anlayabilir miyim?
- Başka bir bölümü etkiliyor mu?
- Nasıl doğrulandığı belli mi?

Matplotlib: bir açıklamanın nereye ekleneceği bile birlikte değerlendirilir.

Kayıt, inceleme ve test farklı işlerdir

Birbirini tamamlarlar. Hiçbiri diğerinin yerine geçmez.

Geçmiş

Ne değişti?

İnceleme

Bu karar anlamlı mı?

Test veya deney

Beklenen davranış gözleniyor mu?

Öğrenci projesinde: ortak teslim

Bir kişi dosya biriktirmek yerine takım aynı projeden devam eder.

- Yazılım, açıklamalar ve ilgili metinler aynı bağlamda.
- Teslimin hangi kayıtlı hâlden geldiği belli.
- Eksikler ve öneriler takım tarafından görülebilir.

Commit sayısı, kişinin emeğinin veya işin kalitesinin ölçüsü değildir.

Araştırmada: yöntemin izini korumak

Bir sonucun hangi kod ve ayarlardan geldiği anlaşılabilsin.

- Analiz kodunun ve ayarların kayıtlı hâli.
- Yöntemdeki değişikliklerin gerekçesi.
- Veri, ortam ve tekrar adımlarına yönelik açıklamalar.

Şirkette: değişikliği kontrollü paylaşmak

Bir kişinin kararı, ürünün geri kalanını etkileyebilir.

Farklı roller

Geliştirme, test, bakım ve ürün ihtiyaçları.

Ortak karar

Değişikliğin etkisi görülerek kabul edilmesi.

Sürdürülebilirlik

Ürünün hangi hâlinin kullanıldığının bilinmesi.

Açık kaynakta: tanımadığın insanla üretmek

Katkı yalnızca kod yazmak değildir.

- Sorun bildirmek ve açıklamayı iyileştirmek de katkıdır.
- Fork: kendi hesabında ilişkili depo kopyası.
- Öneriyi sunmak, kabul edileceği anlamına gelmez.

Arduino Servo: kullanıcı gözlemi → sorun kaydı → belge düzeltme önerisi.

Projeyi sonraki kişiye devretmek

Yeni gelen kişi dosyayı bulsun, kararları anlayabilsin.

- Yeni gelen kişi dosyaların yanında kararların gerekçesini de bulur.
- Proje, onu yapan kişinin sürekli yanında olmasına daha az bağımlı kalır.



README: projenin giriş kapısı

İlk kez gelen biri, nereden başlayacağını anlayabilsin.

- Bu proje ne yapıyor?
- Başlamak için ne gerekiyor?
- Nasıl kullanılır veya denenir?
- Sınırlar ve yardım kanalları nerede?

MDN Web Docs: açıklamayı daha anlaşılır yapmak da katkıdır.

Çalışma kültürü ve ortak sorumluluk

Açık gerekçe ve saygılı iletişim, dosyalar kadar önemlidir.

- Beklenti ve sorumluluk açık olsun.
- Eleştiri kişiye değil, değişikliğe yönelsin.
- Önemli kararın gerekçesi kaybolmasın.

Bedel: öğrenmek ve kayıt tutmak

Başlangıçta daha yavaş hissettirebilir.

Bugün harcanan

Yeni kavramlar, açıklama yazmak, kayıt düzeni.

Sonradan kazanılan

Geçmiş anlamak, değişiklikleri ayırmak, devam etmek.

Bedel: koordinasyon ve bekleme

İnceleme kaliteye katkı verebilir. İş akışına süre de ekler.

- Yorum ve düzeltme için zaman ayrılır.
- Ayrı çalışmalar buluşurken seçim gerekebilir.
- Süreç, ekibin büyüklüğüne ve riskine göre ayarlanır.
- Platformdaki iş takibi ve otomasyonlar da bakım ister.

Görünürlük: paylaşırken sınır koymak

Herkese açık depo, herkesin görebildiği çalışma demektir.

Görünürlük kazancı

Çalışmanın incelenmesi, keşfedilmesi ve paylaşılması.

Paylaşım sorumluluğu

Gizli bilgi, kişisel veri ve erişim anahtarı sınırları.

Güncel dosyadan silmek, yayımlanmış geçmişi kendiliğinden silmez.

Herkese açık depo ve lisans

Lisans, yeniden kullanımın koşullarını açıklar.

- Görünür olmak: içeriğe erişebilmek.
- Lisans: kullanım, değiştirme ve paylaşım koşulları.
- Katkı kuralları: projeye nasıl katılınacağı.

Git hangi sorunları çözmez?

Geçmiş tutmak, bütün mühendislik ihtiyaçlarını kapsamaz.

- Donanım ve ortam ayrıca belgelenir.
- Büyük dosyalar ve CAD için başka yöntemler gerekebilir.
- Testin ve uygun yedekleme düzeninin yerine geçmez.

Ek depolama ve araçlar maliyet oluşturabilir.

Her işte aynı düzen gerekli mi?

Araç ve süreç, projenin ihtiyacına göre seçilir.

Değeri artar

Değişim, işbirliği, geri dönme ve devretme ihtiyacı varsa.

Düzen sadeleşir

Küçük işte temel kayıt. Büyüdükçe uygun ek kurallar.

Bir sonraki projeye üç soru

Git öğrenmeden önce bile çalışma biçimini düşünmeye başlayabilirsin.

- Çalışan hâlimiz hangisi?
- Bu değişikliği neden yaptık?
- Biz olmadan biri devam edebilir mi?

Sorular ve tartışma

Kendi çalışmalarınızda en çok hangi bilgi kayboluyor?

- Git ile GitHub farkı?
- Tek başına kullanmanın değeri?
- Paylaşımın faydası ve bedeli?

Dört temel kelime

Ayrıntıları ezberlemeden, bu kavramları ayırt etmek yeterli.

- Depo / repository: izlenen proje ve geçmişi.
- Commit: projenin belirli bir hâlinin açıklamalı kaydı.
- Dal / branch: aynı geçmişten ayrı bir ilerleyiş.
- Pull request / PR: değişikliği ortak akışa alma önerisi.

Bir README neleri cevaplayabilir?

İlk okuyucunun ihtiyaç duyduğu başlangıç bilgisi.

- Amaç: ne yapar, kimin işine yarar?
- Başlangıç: gerekli ortam ve hazırlık adımları.
- Kullanım: beklenen çıktı ve örnek kullanım.
- Sınırlar: bilinen sorunlar, yardım ve lisans.

Anlaşılır bir değişiklik önerisi

İnceleyen kişinin karar verebilmesi için bağlam.

- İhtiyaç: hangi sorun veya öneri?
- Değişiklik: ne yapıldı, kapsamı ne?
- Doğrulama: nasıl denendi, ne gözlemlendi?
- Açık konu: bilinen sınır veya bekleyen karar?

Projenin ihtiyacına göre seçim

Geçmiş, işbirliği ve dosya türü birlikte değerlendirilir.

Geçmiş ihtiyacı

Önceki hâle dönmek veya değişimi karşılaştırmak gerekiyor mu?

İnsan ihtiyacı

Birlikte çalışacak veya projeyi devralacak başkaları var mı?

Dosya ihtiyacı

Kod ve metin mi, büyük veri veya özel tasarım dosyaları mı?

İşe yarayan dört alışkanlık

Aracın değerini, tekrar edilebilir çalışma düzeni artırır.

- İlişkili değişiklikleri anlaşılır kayıtlarda toplamak.
- Gerekçeyi gelecekteki okuyucu için yazmak.
- Ortak değişiklikleri ihtiyaç kadar incelemek.
- Başlangıç açıklamalarını güncel tutmak.

Kaynaklar ve devam yolu

İhtiyaç duydukça bu kaynaklara ve gerçek projelere dönebilirsin.

[Pro Git kitabı: Git'in temel yaklaşımı](#)

[GitHub Docs: GitHub ve ortak çalışma](#)

[Arduino Servo: bir kullanıcı katkısı](#)

[Matplotlib: birlikte yapılan inceleme](#)

[MDN: dokümantasyona küçük katkı](#)